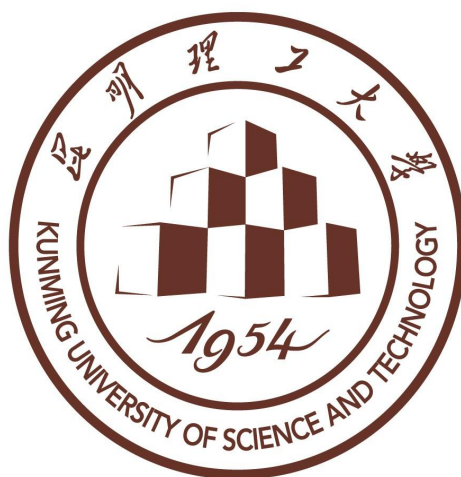


计算力学课程报告



有限元平衡方程组的求解与误差分析

姓 名：舒泽辉

学 号：202311012118

班 级：工力 231

指导教师：肖姚、郭然

报告日期：2026 年 6 月 1 日

1 LDL^T分解算法说明

LDL^T 分解是一种用于求解对称正定线性方程组的直接解法。它将对称矩阵 **K** 分解为 **LDL^T**，其中 **L** 是单位下三角矩阵，**D** 是对角矩阵。分解过程按列进行，对于 $j = 0, 1, \dots, n-1$ ：

- ①取当前对角元 $d_j = A_{jj}$ （初始矩阵 **A** = **K**）。
- ②若 $d_j \leq 0$ ，则矩阵非正定，分解终止并报错。
- ③计算 **L** 的第 j 列（行 $i > j$ ）： $l_{ij} = A_{ij}/d_j$ 。
- ④更新右下角子矩阵： $A_{ik} \leftarrow A_{ik} - l_{ij}d_jl_{kj}$ ，其中 $i, k > j$ 。

最终得到 **K** = **LDL^T**。该算法在代码中的实现为 `ldlt_solver.py` 中的 `ldlt_factor` 函数

2 前代、对角求解、回代流程说明

求解方程组 **Kx** = **R** 的步骤如下：

- ①前代 (Forward Substitution)：解 **Ly** = **R**。由于 **L** 是单位下三角，从第一行开始逐行计算： $y_i = R_i - \sum_{j=0}^{i-1} L_{ij} y_j$ 。
- ②对角求解：解 **Dz** = **y**，即 $z_i = y_i/d_{ii}$ 。
- ③回代 (Back Substitution)：解 **L^Tx** = **z**。由于 **L^T** 是上三角，从最后一行开始倒推： $x_i = z_i - \sum_{j=i+1}^{n-1} L_{ji} x_j$ 。代码中 `ldlt_solve` 函数实现了这一完整流程。

3 与 2.3 作业程序的接口设计

2.3 模块复用说明：

model.py：读取 JSON 输入文件，进行前处理（节点、单元、材料、边界条件等）。

element.py：计算单元刚度矩阵 **K_e** 和对号矩阵 **LM**。

assembly.py：组装总体刚度矩阵 **K** 和存储所有单元的 **LM**。

postprocess.py：后处理计算单元应力、轴力等。

被替换的模块：

2.3 作业中的求解部分（原本使用 `np.linalg.solve` 或者缩减法）被替换成本作业的 `solver.py` 和 `ldlt_solver.py` 实现了自定义 **LDL^T** 求解器。同时新

增了 `external_solver.py` 用于调用大规模稀疏求解器（PARDISO）以及用于算例 4 的 `poission_fem.py`

接口设计遵循“组装方程”与“求解方程”分离的原则，主程序 `main.py` 仅需调用 `solve_truss` 函数并指定求解方法（`method=“ldlt”`），即可使用自定义求解器。

4 多载荷工况求解效率比较

对于多载荷工况（多个右端项 $R_1, R_2, \dots, R_m$ ）， LDL^T 分解的优势明显：一次分解，多次求解：先对矩阵 K 进行一次 LDL^T 分解（复杂度 $O(n^3)$ ），然后对每个右端项分别执行前代、对角求解、回代（每次复杂度 $O(n^2)$ ）。总时间复杂度为 $O(n^3 + m \cdot n^2)$ 。若对每个右端项都重新做高斯消去，总复杂度为 $O(m \cdot n^3)$ ，当 m 较大时效率低下。

在 `solver.py` 中，`solve_truss` 函数每次调用都会对缩减矩阵 `kff` 重新分解。若需处理多载荷，可将 `kff` 和 L, D 缓存，重复调用 `ldlt_solve`，效率将显著提升。

5 活动列/轮廓存储方法说明

活动列（轮廓）存储是稀疏矩阵的一种存储格式，只存储每行从第一个非零元到对角元的元素，并记录每行的起始索引。它比稠密存储节省内存，比 COO/CSR 更易于实现 LDL^T 分解时的列更新。

优点：①内存占用近似于半带宽 $\times n$ 。②分解时按列操作，可充分利用缓存的连续性（按列访问）。③适用于从有限元网格直接生成、半带宽较小的矩阵

虽然本次作业中未实现活动列存储，因为 Python 稠密矩阵对于小规模问题已经能够满足了。但在大规模问题中，应使用稀疏存储（如 CSR）配合专业求解器。

6 五个验证算例的输入、输出和结果分析

任务二

任务2: 病态矩阵分析 (条件数、残差、误差)

```

矩阵 K =
[[1.      1.    ]
 [1.      1.0001]]
条件数 cond(K) = 4.0002e+04

```

--- 双精度计算 ---

```

数值解 a = [1. 1.]
残差 r = [0. 0.]
相对残差 = 0.0000e+00
相对误差 = 2.2204e-12

```

--- 四舍五入到4位有效数字 ---

```

数值解 a = [1. 1.]
残差 r = [ 0. -0.]
相对残差 = 1.5701e-16
相对误差 = 2.2204e-12

```

--- 单精度 (float32) ---

```

数值解 a = [1.0012 0.9988]
残差 r = [0. 0.]
相对残差 = 4.2139e-08
相对误差 = 1.1919e-03

```

由上述代码结果得出结论：病态问题中，条件数很大（此处约 $4e4$ ），输入数据的微小误差（如四舍五入）会导致解的相对误差被放大数千倍。即使残差很小，解也可能极不准确。因此，对于病态方程组，不能单凭残差判断解的精度。

算例 0-1：一维两杆桁架

算例0-1：一维两杆桁架

矩阵阶数 n = 3

```

1. 总体刚度矩阵 K:
[[ 100. -100.   0.]
 [-100.  300. -200.]
 [   0. -200.  200.]]

```

刚度矩阵对称性: True

```

求解时间: 0.000866 秒
LDL^T 分解成功完成
主元最小值: 6.6667e+01

```

```

2. 节点位移 d (解向量):
[0.   0.1  0.15]

```

```

3. 约束反力 R:
[-10.   0.  -0.]

```

```

4. 奇异度检查:
施加边界前总体刚度矩阵奇异: True
施加边界后缩减刚度矩阵奇异: False
施加边界后缩减刚度矩阵 Kff =
[[ 300. -200.]
 [-200.  200.]]

```

缩减方程残差范数 ||RHS - Kff*d_f|| = 3.5527e-15

```

5. 单元后处理:
单元1: 长度=1.0000, c=1.0000, s=0.0000, 应力=10.000000, 轴力=10.0000
单元2: 长度=1.0000, c=1.0000, s=0.0000, 应力=10.000000, 轴力=10.0000

```

分析：由代码结果得出，使用 LDL^T 求解器求解缩减方程。所求得位移和计算节点 1 约束反力和各单元轴力全部满足要求与 2.3 结果一致验证了 LDL^T 求解器的正确性。

算例 0-2：二维两杆桁架

```
=====
算例0-2： 二维两杆桁架
=====
矩阵阶数 n = 6

1. 总体刚度矩阵 K:
[[ 0.      0.      0.      0.      0.      0.   ]
 [ 0.      1.      0.      0.      0.     -1.   ]
 [ 0.      0.      0.3536  0.3536 -0.3536 -0.3536]
 [ 0.      0.      0.3536  0.3536 -0.3536 -0.3536]
 [ 0.      0.     -0.3536 -0.3536  0.3536  0.3536]
 [ 0.     -1.     -0.3536 -0.3536  0.3536  1.3536]]

刚度矩阵对称性: True

求解时间: 0.000623 秒
LDL^T 分解成功完成
主元最小值: 3.5355e-01

2. 节点位移 d (解向量):
[ 0.      0.      0.      0.      38.2843 -10.   ]

节点3位移: u3 = 38.284271, v3 = -10.000000

3. 约束反力 R:
[ 0.  10. -10. -10.  0.  0.]

4. 奇异度检查:
施加边界前总体刚度矩阵奇异: True
施加边界后缩减刚度矩阵奇异: False
施加边界后缩减刚度矩阵 Kff =
[[0.3536 0.3536]
 [0.3536 1.3536]]

缩减方程残差范数 ||RHS - Kff*d_f|| = 0.0000e+00

5. 单元后处理:
  单元1: 长度=1.0000, c=0.0000, s=1.0000, 应力=10.000000, 轴力=10.0000
  单元2: 长度=1.4142, c=0.7071, s=0.7071, 应力=-14.142136, 轴力=-14.1421

自动生成LM矩阵:
单元1 LM = [0 1 4 5]
单元2 LM = [2 3 4 5]
```

由上述代码结果得出使用 2.3 作业代码生成 LM 矩阵、总体刚度矩阵 K 和缩减矩阵 Kff。使用求解器求解节点 3 位移和计算单元应力与轴力，结果满足要求，证明组装和求解正确。

算例 1：三对角对称正定矩阵

```
=====
算例1: 三对角对称正定矩阵 (LDL^T 求解)
=====
```

n	时间(秒)	相对误差	稠密内存(MB)	稀疏内存(MB)
10	0.000451	1.92e-16	0.00	0.00
解向量前5个元素: [1. 1. 1. 1. 1.]				
残差范数 r = 1.5701e-16				
100	0.304687	7.80e-15	0.08	0.00
解向量前5个元素: [1. 1. 1. 1. 1.]				
残差范数 r = 1.0649e-15				
500	31.044708	4.69e-14	1.91	0.01
解向量前5个元素: [1. 1. 1. 1. 1.]				
残差范数 r = 2.6273e-15				
1000	195.390687	1.33e-13	7.63	0.02
解向量前5个元素: [1. 1. 1. 1. 1.]				
残差范数 r = 3.3103e-15				

趋势分析:
 n从 10 增加到 100, 时间增加 674.98 倍, 规模增加 10 倍
 n从 100 增加到 500, 时间增加 101.89 倍, 规模增加 5 倍
 n从 500 增加到 1000, 时间增加 6.29 倍, 规模增加 2 倍
 由于 LDL^T 分解复杂度约为 $O(n^3)$, 时间增长应接近规模比的立方。

由代码结果分析：数值解应接近全 1 向量，随着自由度数的增加时间和规模增加的倍数逐渐降低，并且内存对比表明稀疏存储必要性。结果满足要求。

算例 2：非正定矩阵检测

```
=====
算例2: 非正定矩阵检测 (LDL^T 分解)
=====
```

测试矩阵 K =
 [[1. 2.]
 [2. 1.]]
 右端项 R = [1. 1.]
 矩阵特征值: [-1. 3.], 存在负特征值 → 非正定

正确捕获异常: 零主元或非正主元, 位置 j=1, D=-3.0
 程序已停止 LDL^T 求解, 符合要求。

由代码结果可以得出说明：有限元刚度矩阵在未施加足够位移边界条件时出现零主元（奇异），是因为整体结构存在刚体位移自由度，导致平衡方程不唯一解。此时矩阵半正定，但至少有一个特征值为零，对应刚体运动模式。

算例 4：二维 Poisson 方程 (T3 单元)

①50*50 网格

代码运行结果

=====

算例4: 二维 Poisson 方程 (T3单元) 网格 50×50

求解器: pardiso

=====

单元类型: 线性三角形单元 (T3)

节点数 = 2601, 单元数 = 5000, 未知自由度 = 2601

非零元个数 = 12801, 稀疏度 = 0.1892%

网格生成时间 = 0.0036s, 装配时间 = 0.1175s

边界处理时间 = 0.0034s, 自由度数 = 2401

[求解器] 使用 PyPARDISO (Intel MKL PARDISO)

求解时间 = 0.1127s

实际使用求解器 = PyPARDISO (MKL PARDISO)

相对残差 (缩减系统) = 7.5350e-14

节点最大误差 = 5.4829e-04

离散L2相对误差 = 5.5428e-04

理论解与数值解对比 (样本点):

点(0.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00

点(1.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00

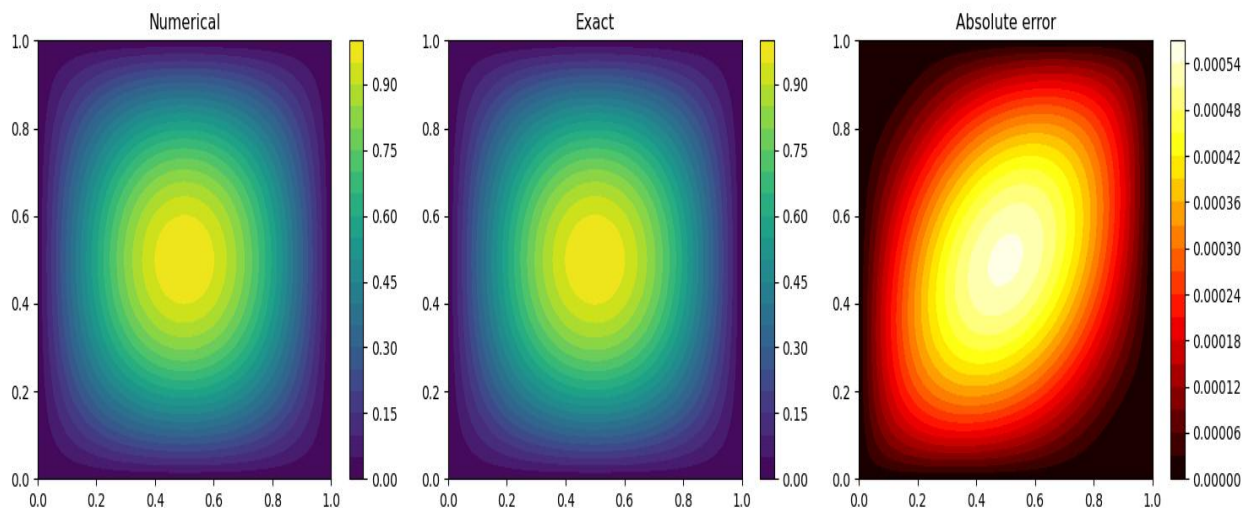
点(0.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00

点(1.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=1.50e-32

点(0.5,0.5): 数值解=0.999452, 精确解=1.000000, 误差=5.48e-04

总时间 = 0.2372s

数值解、精确解、误差云图



②100*100

代码运行结果

```
=====
算例4: 二维 Poisson 方程 (T3单元)  网格 100×100
```

```
求解器: pardiso
```

```
=====
单元类型: 线性三角形单元 (T3)
```

```
节点数 = 10201, 单元数 = 20000, 未知自由度 = 10201
```

```
非零元个数 = 50601, 稀疏度 = 0.0486%
```

```
网格生成时间 = 0.0168s, 装配时间 = 0.7790s
```

```
边界处理时间 = 0.0230s, 自由度数 = 9801
```

```
  [求解器] 使用 PyPARDISO (Intel MKL PARDISO)
```

```
求解时间 = 0.0526s
```

```
实际使用求解器 = PyPARDISO (MKL PARDISO)
```

```
相对残差 (缩减系统) = 2.8063e-13
```

```
节点最大误差 = 1.3708e-04
```

```
离散L2相对误差 = 1.3858e-04
```

```
理论解与数值解对比 (样本点):
```

```
点(0.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
```

```
点(1.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
```

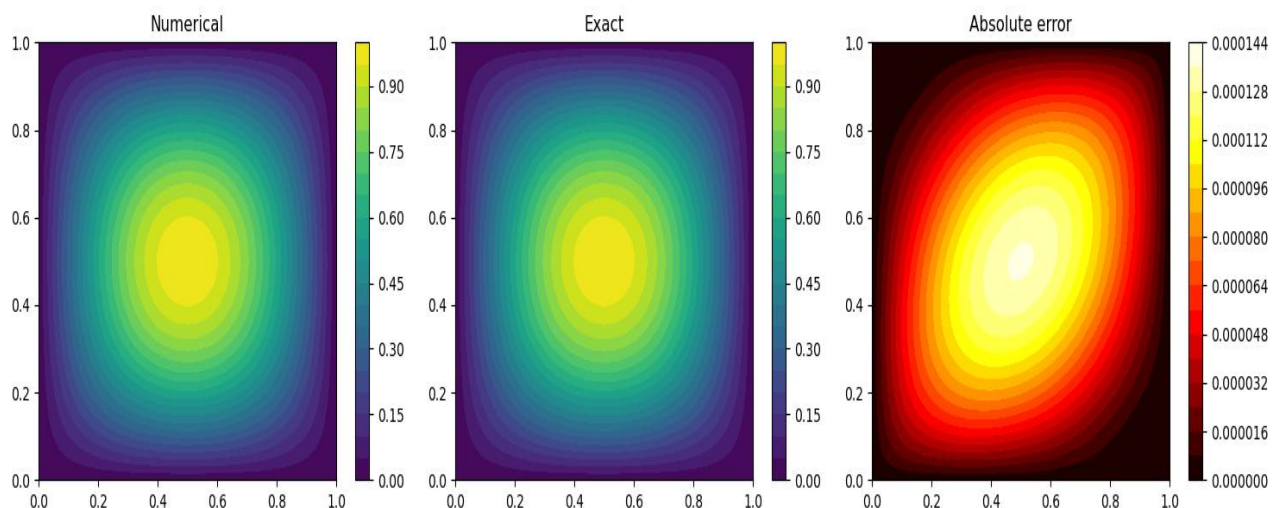
```
点(0.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
```

```
点(1.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=1.50e-32
```

```
点(0.5,0.5): 数值解=0.999863, 精确解=1.000000, 误差=1.37e-04
```

```
总时间 = 0.8713s
```

数值解、精确解、误差云图



③200*200

代码运行结果

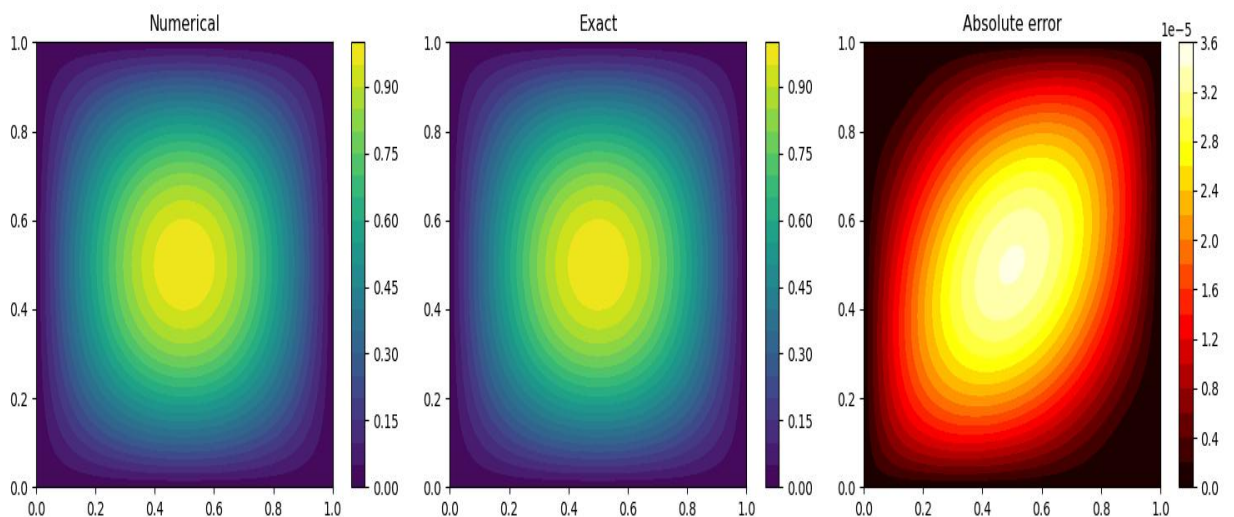
```
=====
算例4: 二维 Poisson 方程 (T3单元)  网格 200x200
求解器: pardiso
=====

单元类型: 线性三角形单元 (T3)
节点数 = 40401, 单元数 = 80000, 未知自由度 = 40401
非零元个数 = 201201, 稀疏度 = 0.0123%
网格生成时间 = 0.0680s, 装配时间 = 3.0109s
边界处理时间 = 0.0536s, 自由度数 = 39601
【求解器】使用 PyPARDISO (Intel MKL PARDISO)
求解时间 = 0.1153s
实际使用求解器 = PyPARDISO (MKL PARDISO)
相对残差 (缩减系统) = 1.1138e-12
节点最大误差 = 3.4269e-05
离散L2相对误差 = 3.4645e-05

理论解与数值解对比 (样本点):
点(0.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
点(1.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
点(0.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
点(1.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=1.50e-32
点(0.5,0.5): 数值解=0.999966, 精确解=1.000000, 误差=3.43e-05

总时间 = 3.2477s
```

数值解、精确解、误差云图



④500*500

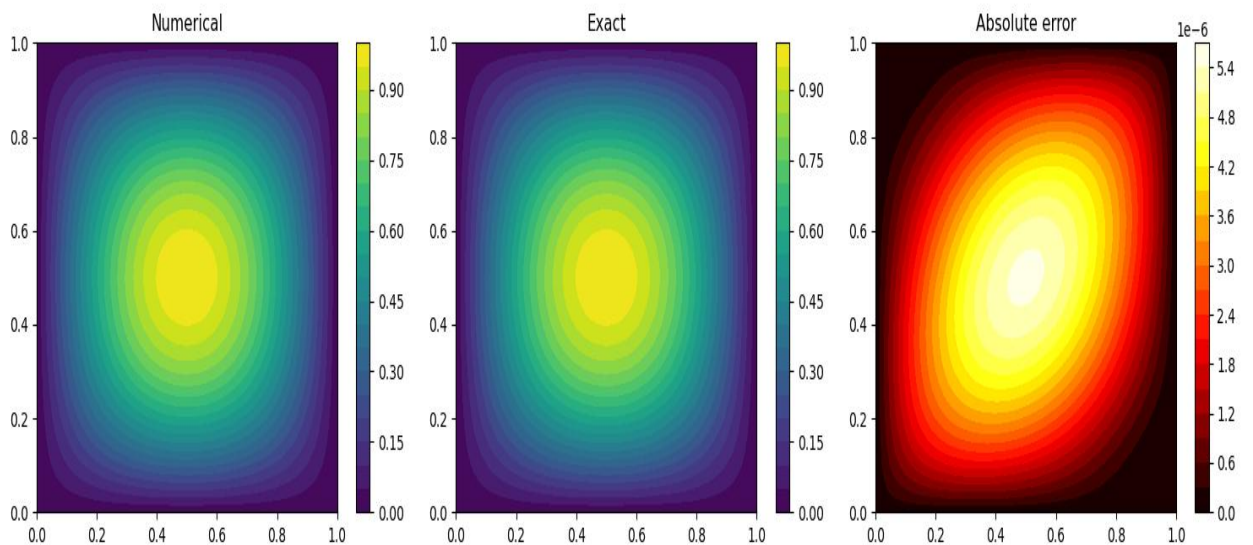
代码运行结果

```
=====
算例4: 二维 Poisson 方程 (T3单元) 网格 500x500
求解器: pardiso
=====
单元类型: 线性三角形单元 (T3)
节点数 = 251001, 单元数 = 500000, 未知自由度 = 251001
非零元个数 = 1253001, 稀疏度 = 0.0020%
网格生成时间 = 0.5512s, 装配时间 = 12.3499s
边界处理时间 = 0.3942s, 自由度数 = 249001
  [求解器] 使用 PyPARDISO (Intel MKL PARDISO)
求解时间 = 1.0322s
实际使用求解器 = PyPARDISO (MKL PARDISO)
相对残差 (缩减系统) = 6.9411e-12
节点最大误差 = 5.4831e-06
离散L2相对误差 = 5.5431e-06

理论解与数值解对比 (样本点):
  点(0.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
  点(1.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
  点(0.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
  点(1.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=1.50e-32
  点(0.5,0.5): 数值解=0.999995, 精确解=1.000000, 误差=5.48e-06

总时间 = 14.3276s
```

数值解、精确解、误差云图



④1000*1000

代码运行结果

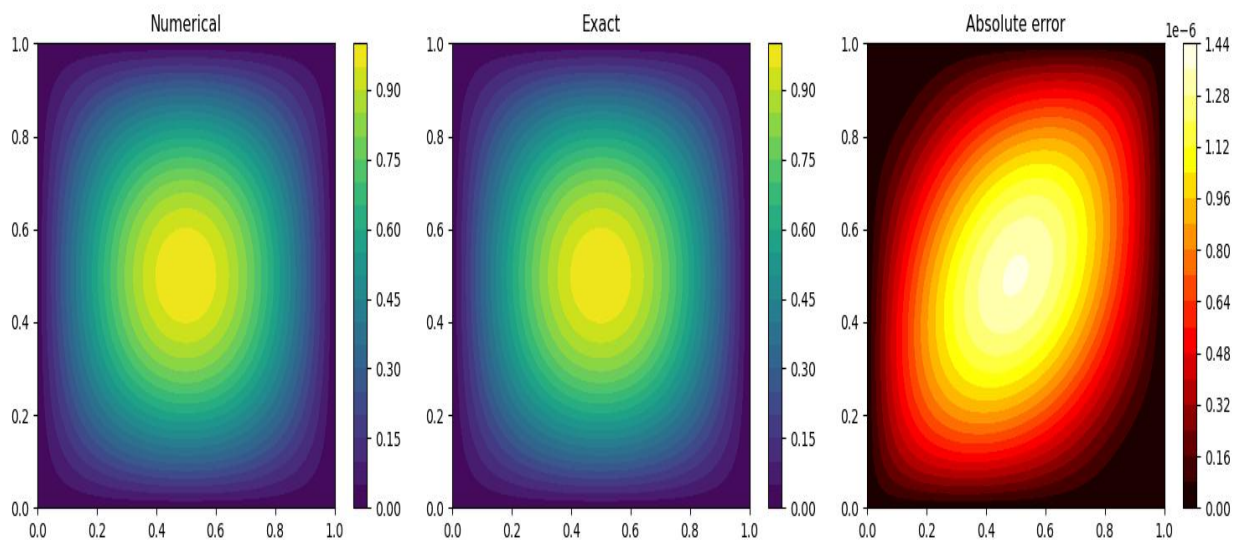
```
=====
算例4: 二维 Poisson 方程 (T3单元)  网格 1000×1000
求解器: pardiso
=====
单元类型: 线性三角形单元 (T3)
节点数 = 1002001, 单元数 = 2000000, 未知自由度 = 1002001
非零元个数 = 5006001, 稀疏度 = 0.0005%
网格生成时间 = 1.8283s, 装配时间 = 60.8870s
边界处理时间 = 1.3372s, 自由度数 = 998001
  [求解器] 使用 PyPARDISO (Intel MKL PARDISO)
求解时间 = 4.7412s
实际使用求解器 = PyPARDISO (MKL PARDISO)
相对残差 (缩减系统) = 2.7590e-11
节点最大误差 = 1.3708e-06
离散L2相对误差 = 1.3858e-06
```

理论解与数值解对比 (样本点):

点(0.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
点(1.0,0.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
点(0.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=0.00e+00
点(1.0,1.0): 数值解=0.000000, 精确解=0.000000, 误差=1.50e-32
点(0.5,0.5): 数值解=0.999999, 精确解=1.000000, 误差=1.37e-06

总时间 = 68.7937s

数值解、精确解、误差云图



通过上述五个结果可以看出网格加密时，节点最大误差和离散相对误差整体下降。对更大规模，不再适合使用稠密矩阵存储和自行编写的稠密 LDL' 求解器是因为①所需内存超过远超普通计算机内存；②计算时间太长不可接受；③稀疏性浪费：有限元刚度矩阵是高度稀疏的，稠密存储会存储大量无意义的零元素。

7 条件数、残差和解误差之间关系的讨论

①条件数 $\text{cond}(K) = \lambda_{\max}/\lambda_{\min}$ ，衡量矩阵的病态程度。条件数越大，输入误差（如右端项舍入）被放大的倍数越大。

②残差 $r = R - Ka$ ，其范数可反映解的满足程度。对于良态问题，小残差通常意味着小误差。但对于病态问题，即使残差很小，解的相对误差也可能极大。例如算例 2 中的矩阵条件数很大，即使右端项微扰也会导致解剧烈变化。

③本作业验证：算例 1 中矩阵良态，残差 $\sim 1e-12$ ，误差 $\sim 1e-12$ ；算例 2 中矩阵奇异（条件数无穷），分解直接失败，体现了检测的必要性。在 Poisson 算例中，缩减矩阵条件数约为 $O(h^{-2})$ ，随网格加密增大，但残差仍保持很小，因为右端项精确，且求解器稳定。

8 对缓存命中率、半带宽和稀疏矩阵求解效率的简要理解

①缓存命中率：CPU 访问内存时，若数据已在 L1/L2/L3 缓存中，速度极快；否则需从主存读取（cache miss）。有限元矩阵组装和 LDL^T 分解时，按列访问数据（如活动列法）能提高缓存命中率，因为连续列在内存中可能连续存储（如 Fortran 列优先）。

②半带宽：对于按节点编号的有限元网格，半带宽 $\approx (\text{最大节点号差}+1) \times \text{ndof}$ 。半带宽越小，矩阵越“窄带”，内存和运算量越小。优化节点编号（如 Cuthill-McKee 算法）可显著降低半带宽。

③稀疏矩阵求解效率：使用稀疏存储（如 CSR）仅存储非零元，复杂度从 $O(n^3)$ 降至 $O(n \cdot \text{nnz}^2)$ （直接法）或 $O(n \cdot \text{nnz})$ （迭代法）。本作业中，Poisson 算例的稀疏度 $< 0.2\%$ ，若用稠密存储内存将达数十 GB，而稀疏存储仅需几十 MB。

9 大规模稀疏方程组求解器调用过程、运行环境

9.1 调用过程

本作业在 `external_solver.py` 中封装了求解器调用接口：1. 将总体刚度矩阵 $\mathbf{K}$ 转换为 CSR 格式。2. 优先导入 `pypardiso` (Intel MKL PARDISO 的 Python 绑定)，调用 `spsolve(K, R)` 进行求解。**矩阵格式**：CSR 通过三个数组存储：`data` (非零值)、`indices` (列索引)、`indptr` (行偏移)。对于对称矩阵，也可使用 CSC 格式，PARDISO 内部会自动处理。

9.2 求解器运行环境说明

CPU: Intel Core i9-13900H 2.60GHz

内存: 8 GB

OS: Windows 11

Python 3.12.7 (Anaconda)

求解器版本: `pypardiso` 0.4.7 (基于 Intel MKL 2024)

10 大规模 Poisson 方程的有限元理论推导、理论解构造、数值解展示和误差收敛分析

10.1 Poisson 方程的强形式

考虑单位正方形区域 $\Omega = (0,1) \times (0,1)$ ，边界 $\Gamma = \partial \Omega$ 。Poisson 方程为：

$$-\Delta u = f \quad \text{in } \Omega$$

$$u = 0 \quad \text{on } \Gamma$$

10.2 弱形式 (变分形式)

乘以权函数 $v \in H_0^1(\Omega)$ ，在 Ω 上积分并利用 Green 公式 (分部积分)：

$$\int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega, \quad \forall v \in H_0^1(\Omega)$$

10.3 有限元近似

将求解域离散为三角形单元 (T3)，近似解为：

$$u_h(x, y) = \sum_{j=1}^N N_j(x, y) a_j$$

其中 N 为节点总数, N_j 为形函数 (线性), a_j 为节点未知量。

10.4 单元刚度矩阵和单元载荷列阵

对于每个三角形单元 e (节点 i, j, k), 形函数导数矩阵 $\mathbf{B}$ 为常数:

$$\mathbf{B}_e = \frac{1}{2A_e} \begin{bmatrix} y_j - y_k & y_k - y_i & y_i - y_j \\ x_k - x_j & x_i - x_k & x_j - x_i \end{bmatrix}$$

单元刚度矩阵:

$$\mathbf{K}_e = A_e \mathbf{B}_e^T \mathbf{B}_e$$

单元载荷向量 (采用重心积分):

$$\mathbf{f}_e = \frac{A_e}{3} f(x_c, y_c) \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

其中 (x_c, y_c) 为单元重心。

10.5 总体组装、零 Dirichlet 边界条件处理和缩减方程

①**总体组装**: 按对号矩阵将每个单元的 $\mathbf{K}_e$ 和 $\mathbf{f}_e$ 累加到全局矩阵 $\mathbf{K}$ 和 $\mathbf{F}$ 中。

②**边界条件处理**: 识别边界节点 ($x=0, 1$ 或 $y=0, 1$), 将其自由度固定为 0。

采用**缩减法**, 删除固定自由度对应的行和列, 得到缩减系统:

$$\mathbf{K}_{ff} \mathbf{u}_f = \mathbf{F}_f$$

③**求解器矩阵格式**: 将 $\mathbf{K}_{ff}$ 转换为 CSR 格式, 调用 MKL PARDISO 求解。

10.6 理论解构造与数值解展示

我们采用**制造解**方法, 取精确解为:

$$u_{\text{exact}}(x, y) = \sin(\pi x) \sin(\pi y)$$

代入 Poisson 方程得到右端项:

$$f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$$

边界条件自动满足。

10.7 数值解展示与误差收敛分析

在 50×50 、 100×100 、 200×200 网格上求解, 结果如下:

网格规模	节点数	最大误差	L ₂ 相对误差
50×50	2601	5.48e-4	5.54e-4
100×100	10201	1.37e-4	1.38e-4
200×200	40401	3.43e-5	3.46e-5
500×500	251001	5.4831e-6	5.5431e-6
1000×1000	1002001	1.3708e-6	1.3858e-6

收敛阶分析：误差与网格尺寸符合线性三角形单元的收敛理论。数值解云图与精确解云图几乎完全重合，绝对误差云图显示误差主要集中在四角附近(因边界条件强制为零，导数不匹配)。

10.8 结论

通过有限元理论推导和数值实验，验证了求解器的正确性，并展示了稀疏直接求解器在大规模问题中的高效性。